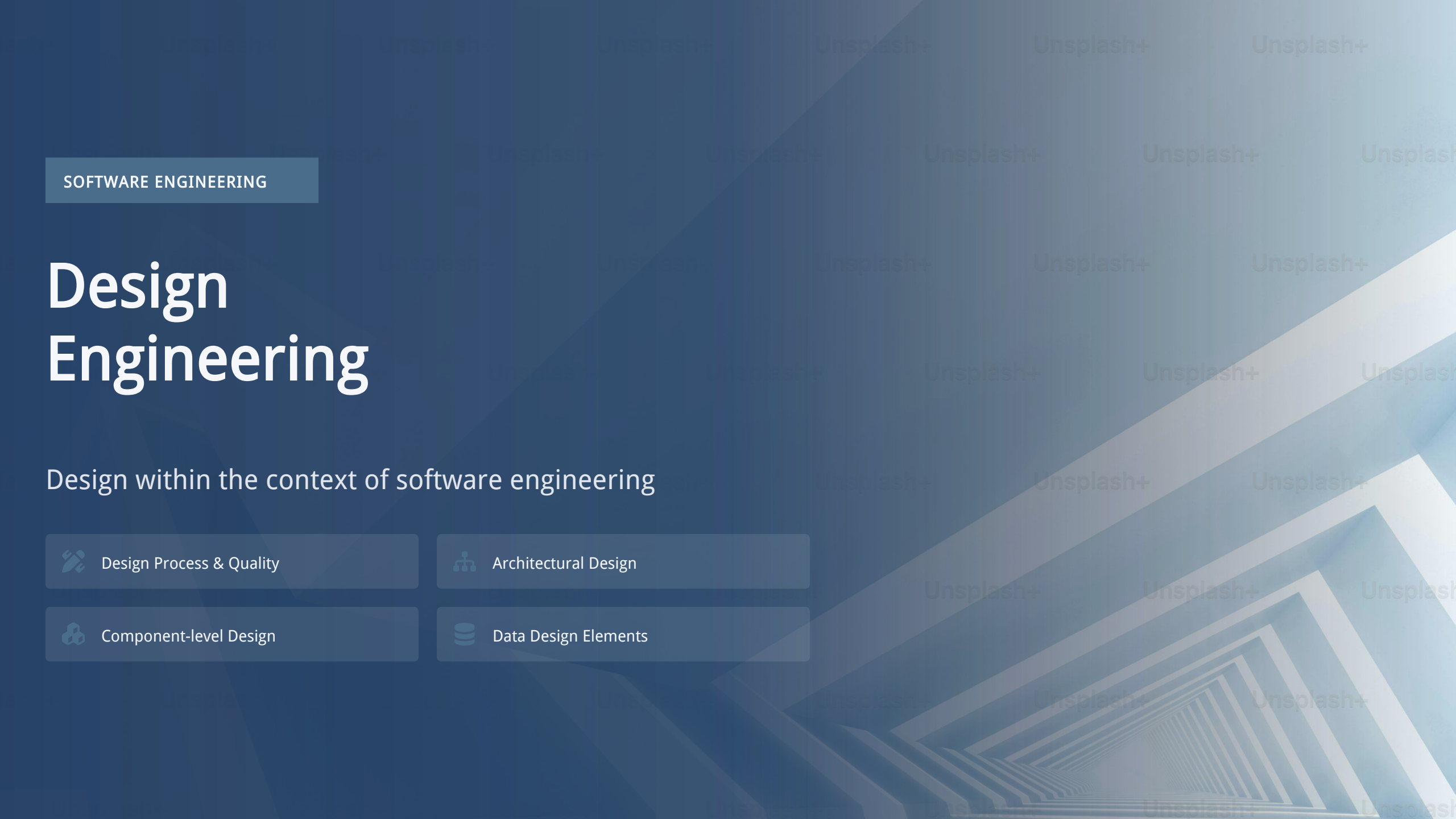
SOFTWARE ENGINEERING

Design Engineering

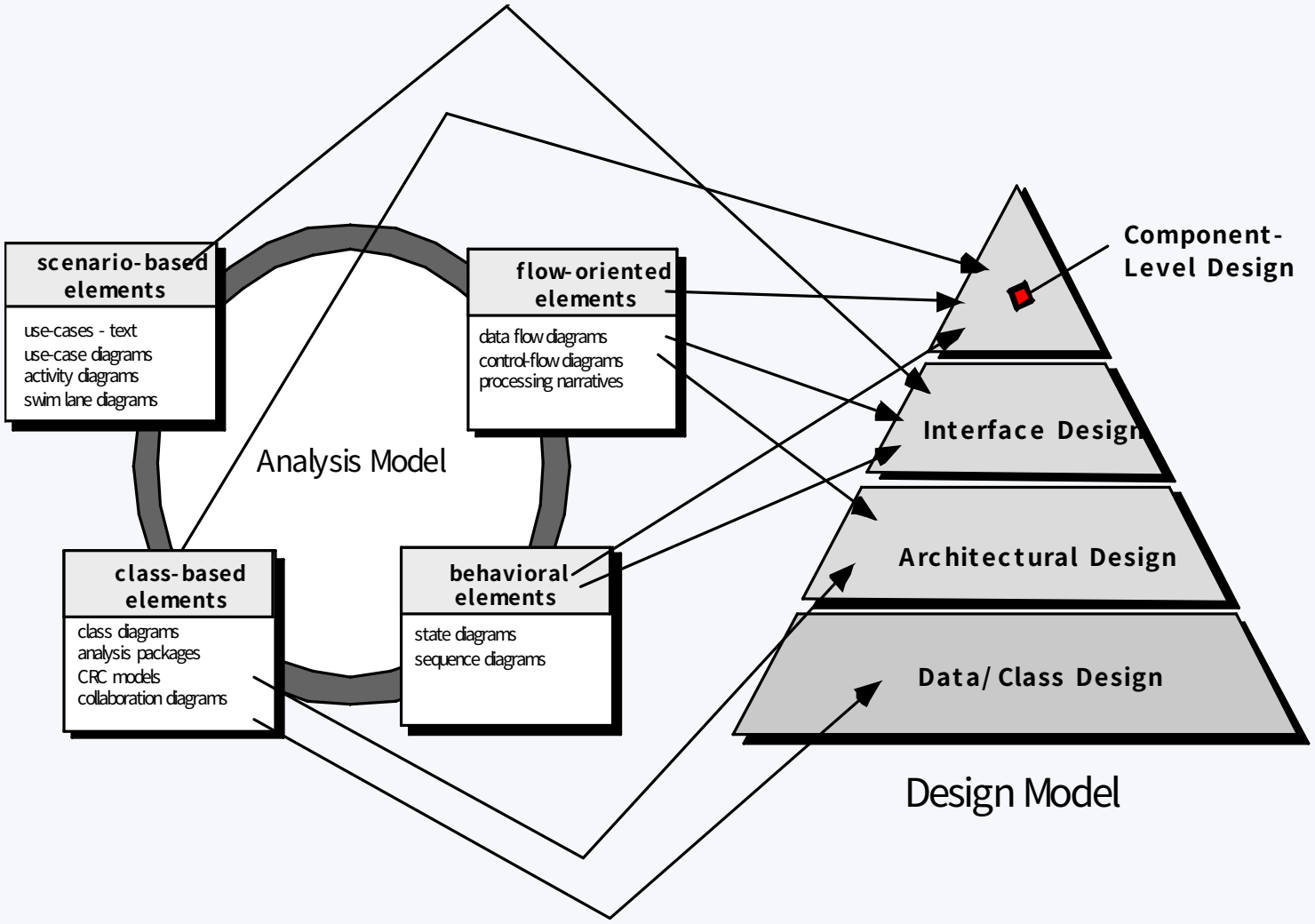
Design within the context of software engineering

- Design Process & Quality
- Architectural Design
- Component-level Design
- Data Design Elements



Design Transformation

Analysis Model → Design Model



Transformation Process

1. Map analysis classes to design classes
2. Refine relationships and collaborations
3. Define architectural patterns
4. Specify interfaces and components
5. Create deployment model

Design Principles

Design and Quality

Requirement Implementation

The design must implement all explicit requirements from the analysis model and accommodate all implicit requirements desired by the customer.

Readability & Understanding

The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.

Complete Picture

The design should provide a complete picture of the software , addressing data, functional, and behavioral domains from an implementation perspective.

Key Insight

Quality design serves multiple audiences: developers need implementation clarity, testers need verification criteria, and maintainers need architectural understanding for future evolution.

Quality Guidelines (Part 1)

Architectural Structure

A design should exhibit an architecture that has been created using recognizable architectural styles or patterns.

Benefit: Provides proven solutions and common vocabulary for communication

Example: Layered architecture, MVC pattern, microservices

Component Characteristics

The architecture is composed of components that exhibit good design characteristics such as high cohesion and low coupling.

Cohesion: Single-minded, focused functionality

Coupling: Minimal dependencies between components

Evolutionary Implementation

The design can be implemented in an evolutionary fashion, allowing for iterative development and incremental delivery.

Modularity

The software should be logically partitioned into elements or subsystems for better organization and maintenance.

Note: For smaller systems, design can sometimes be developed linearly.

Design Quality

Quality Guidelines (Part 2)

Distinct Representations

A design should contain distinct representations of data, architecture, interfaces, and components.

Interface Design

Design should lead to interfaces that reduce complexity of connections between components and with the external environment.

Data Structures

Design should lead to data structures appropriate for classes to be implemented and drawn from recognizable patterns.

Repeatable Method

Design should be derived using a repeatable method driven by information obtained during software requirements analysis.

Component Independence

Design should lead to components that exhibit independent functional characteristics for better maintainability.

Notation

Design should be represented using a notation that effectively communicates its meaning (e.g., UML diagrams).

> **Design Principle:** Good design balances multiple quality attributes while maintaining clarity, modularity, and implementability.

Quality Framework

Quality Attributes - FURPS

Proposed by HP as a comprehensive quality framework

F

Functionality

The system's ability to perform required functions correctly and completely.

- Feature completeness
- Correctness
- Security

U

Usability

How easily users can learn and use the system effectively.

- Learnability
- User satisfaction
- Accessibility

R

Reliability

The system's ability to maintain performance under stated conditions.

- Availability
- Fault tolerance
- Recoverability

P

Performance

How well the system meets performance requirements like speed and efficiency.

- Response time
- Throughput
- Resource usage

S

Supportability

The ability to maintain and enhance the system over its lifetime.

- Testability
- Maintainability
- Scalability

Balance: Quality attributes often involve trade-offs (e.g., performance vs. maintainability)

Source: HP Quality Framework

Design Process

Generic Task Set for Design

1 Data Structures

Design data structures for efficient information storage and retrieval.

2 Architectural Styles

Select appropriate architectural patterns for the system's overall organization.

3 Subsystem Partitioning

Partition the system into manageable, cohesive subsystems.

4 Design Classes

Create a set of design classes (components) with defined responsibilities.

5 External Interfaces

Design interfaces for communication with external systems and services.

6 User Interfaces

Design user interfaces focusing on usability and user experience.

7 Component-level Design

Detailed design of individual components and their internal logic.

8 Deployment Model

Develop a deployment model specifying physical distribution.

Process Note: These tasks are iterative and often overlap during the design process

Adapted from Software Engineering Standards

Design Foundations

Fundamental Design Concepts

Abstraction

- Data
- Procedure
- Control

Architecture

The overall structure of the software system

Patterns

Conveys the essence of a proven design solution

Modularity

Compartmentalization of data and function

Information Hiding

Controlled interfaces to module internals

Functional Independence

Single-minded function and low coupling

Refinement

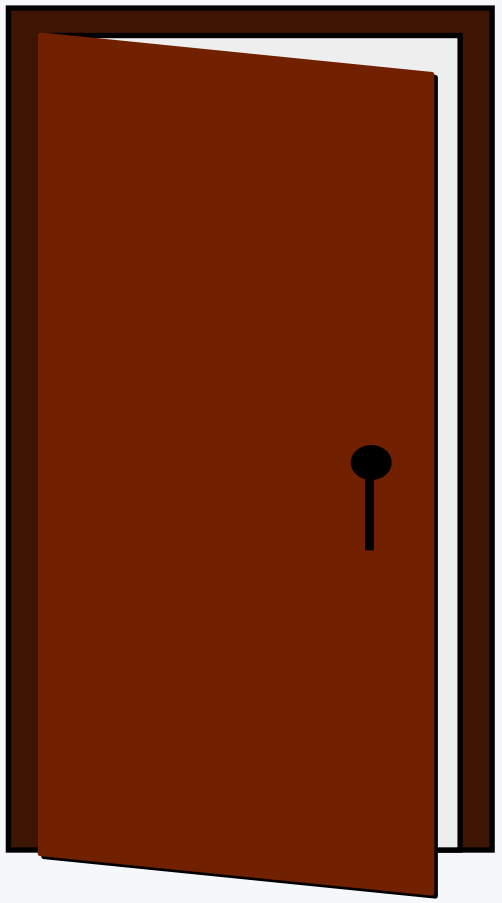
Elaboration of detail for all abstractions

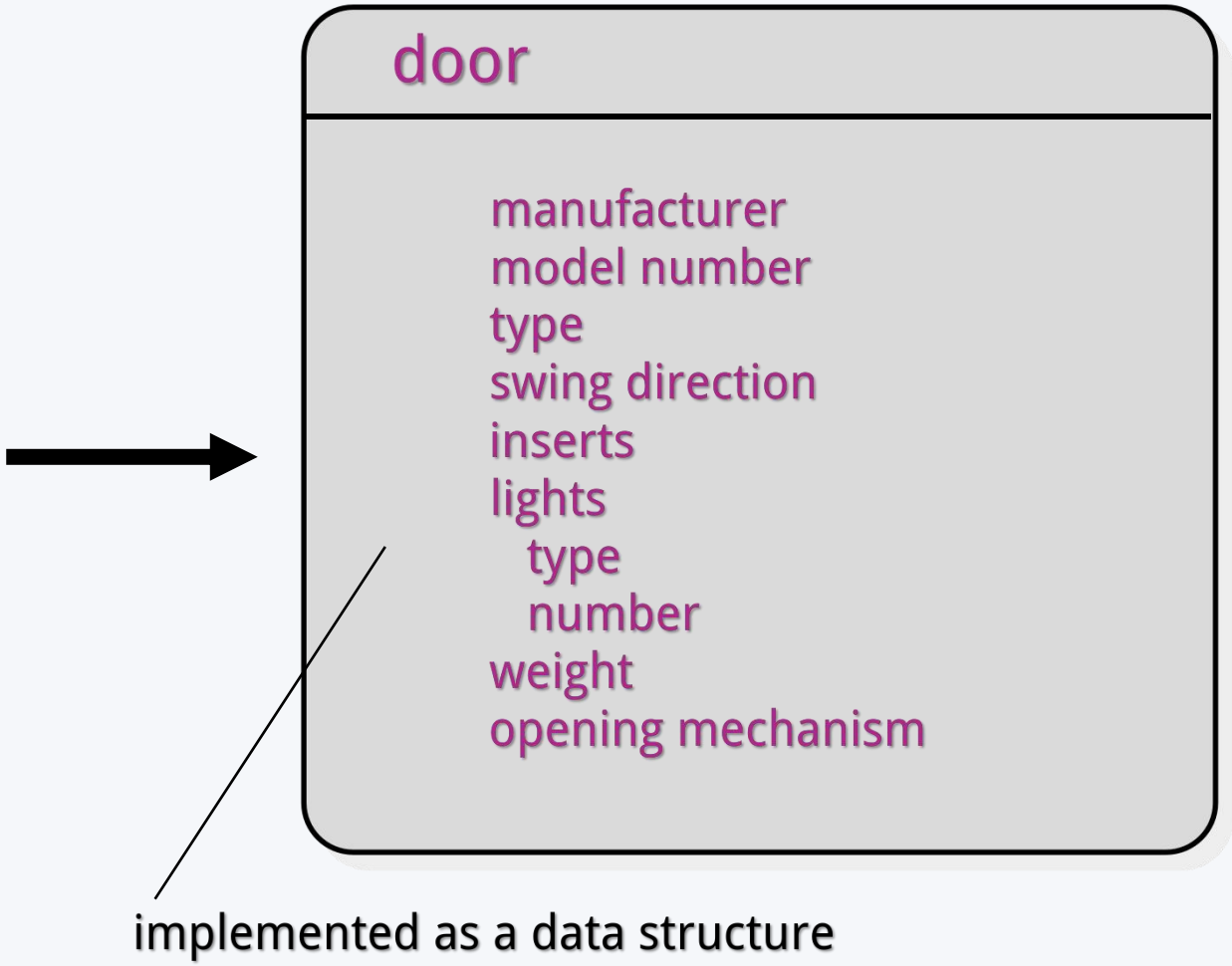
Refactoring

A reorganization technique that simplifies the design

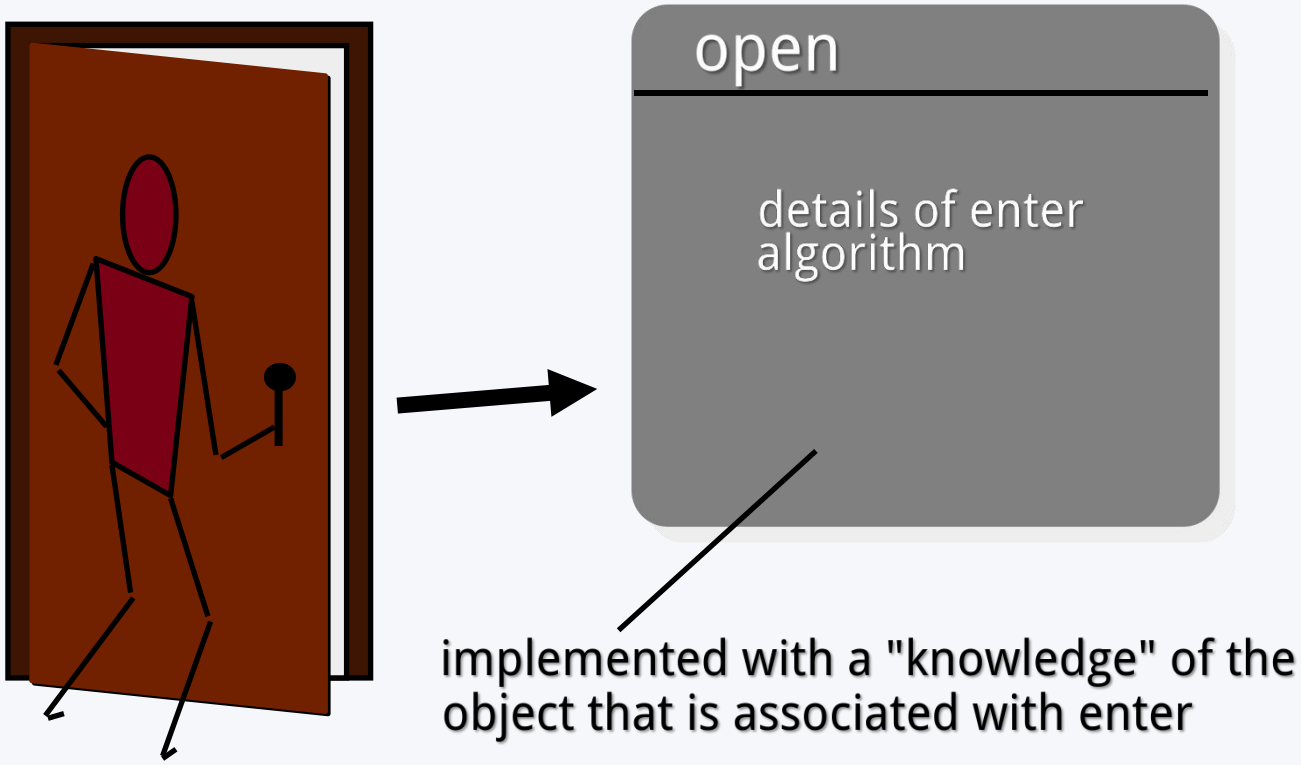
Core Principle: These concepts work together to create maintainable, understandable, and flexible software designs.

Data Abstraction





Procedural Abstraction



Key Concept

Procedural abstraction hides complex implementation behind a simple interface, allowing users to interact with the concept of "opening a door" without needing to understand the detailed steps involved.

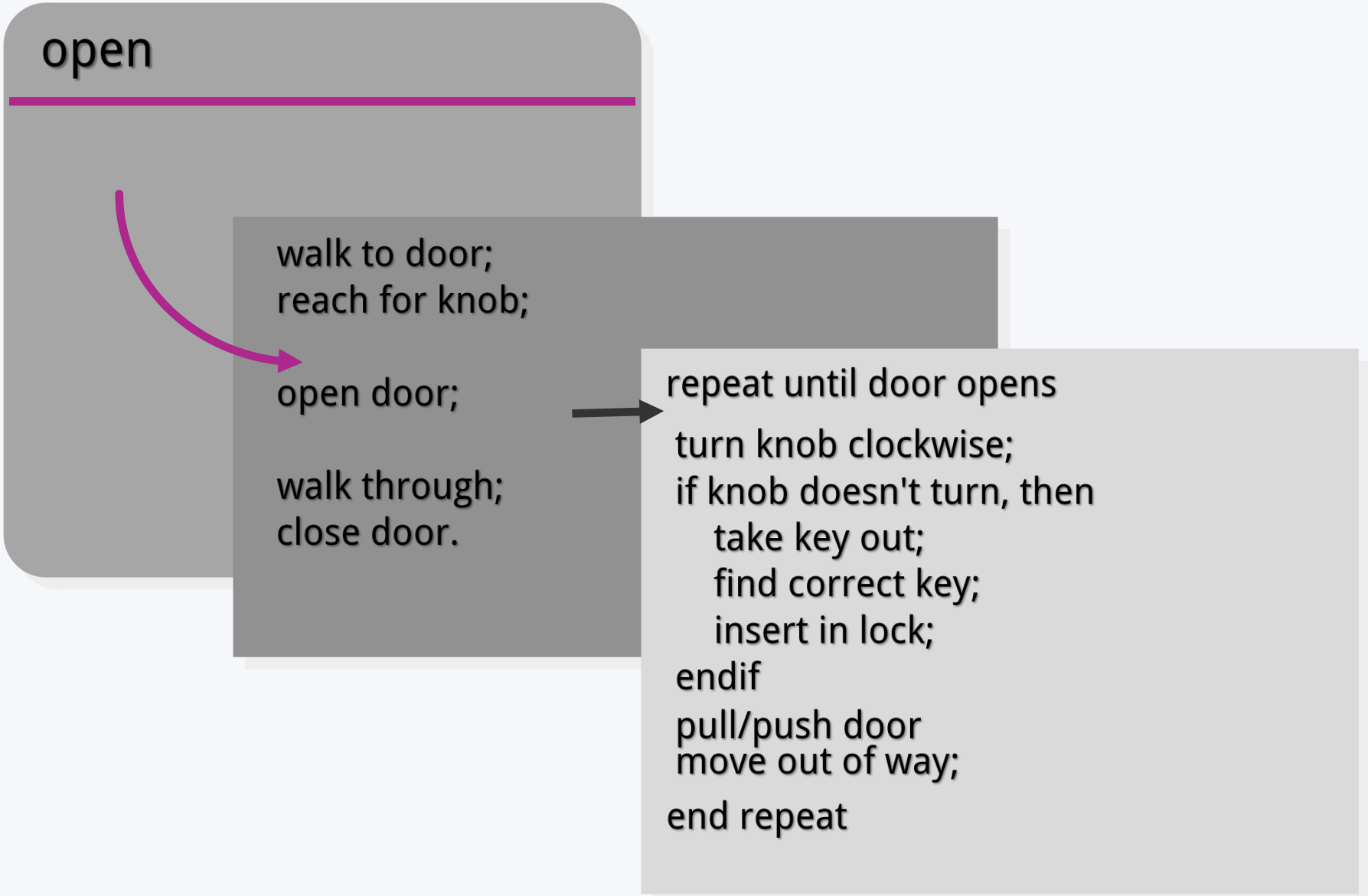
Benefits

- ✓ Simplifies usage
- ✓ Reduces complexity
- ✓ Enables reusability
- ✓ Allows implementation changes

Design Process

Stepwise Refinement

Breaking down high-level procedures into detailed steps



Reusable Solutions

Design Pattern Template

Standardized format for documenting proven design solutions

1 Pattern Name

Describes the essence in a short but expressive name

2 Intent

Describes the pattern and what it does

3 Also-Known-As

Lists any synonyms for the pattern

4 Motivation

Provides an example of the problem

5 Applicability

Notes specific design situations where applicable

6 Structure

Describes required classes for implementation

7 Participants

Describes responsibilities of required classes

8 Collaborations

How participants collaborate to fulfill responsibilities

9 Consequences

Design forces and potential trade-offs

10 Related Patterns

Cross-references related design patterns

Design Principles

Modular Design

What is Modularity?

Modularity is the property of a system that has been decomposed into a set of cohesive and loosely coupled modules.

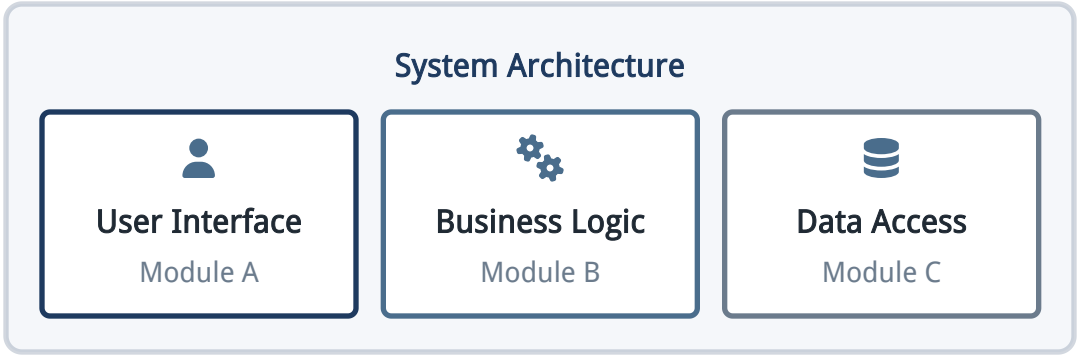
Key Benefits

- ✓ Easier to understand and maintain
- ✓ Supports parallel development
- ✓ Enables reusability
- ✓ Isolates changes and errors

Design Principle

Each module should have a single, well-defined purpose and minimal dependencies on other modules.

Modular System Architecture



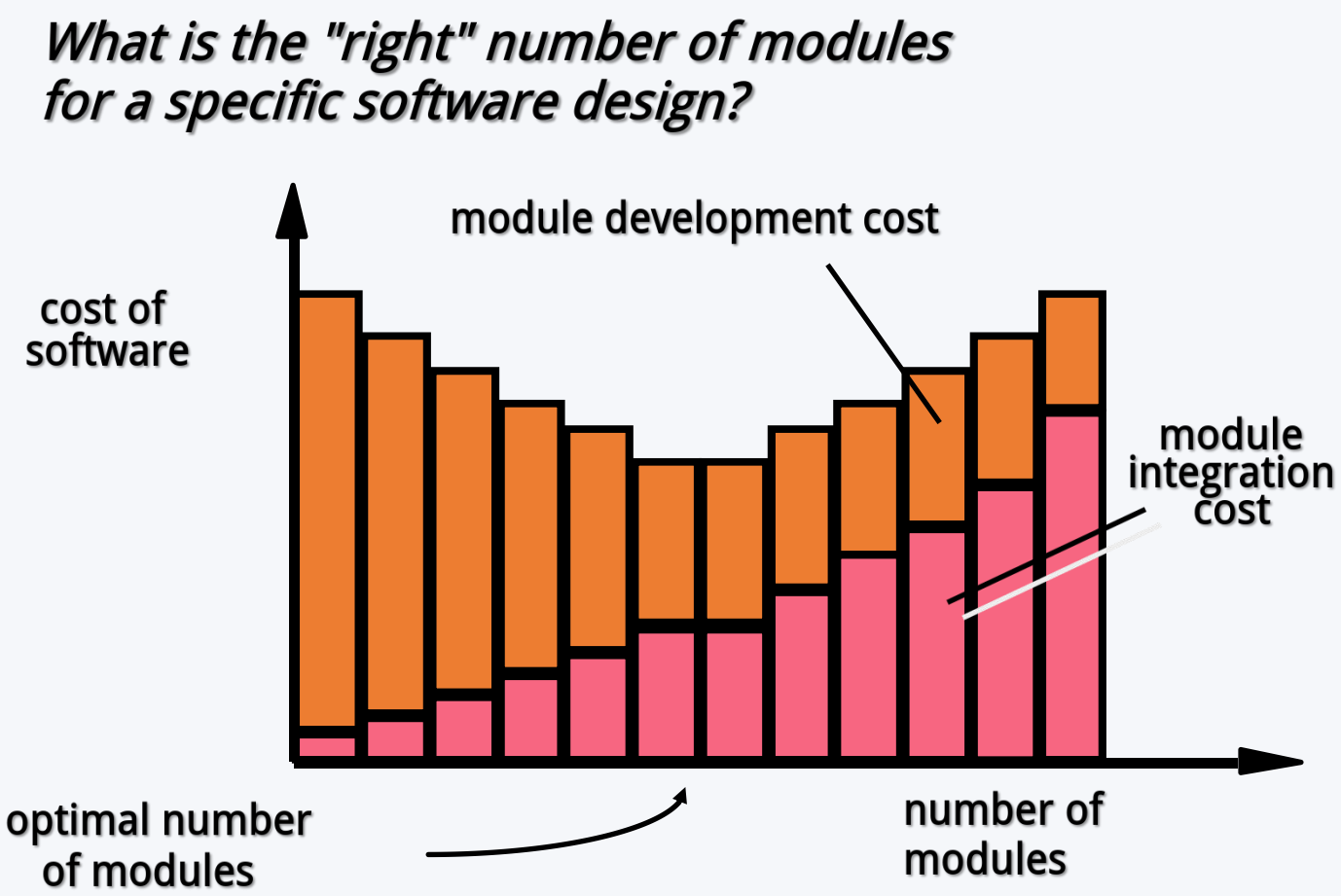
Characteristics of Good Modules

- ★ **High Cohesion** — Single-focused purpose
- ★ **Low Coupling** — Minimal dependencies
- ★ **Well-Defined Interface** — Clear API contract
- ★ **Information Hiding** — Internal details concealed

Design Decisions

Modularity: Trade-offs

What is the "right" number of modules?



Development Cost

Module development cost decreases as the number of modules increases because each module is smaller and simpler to build.

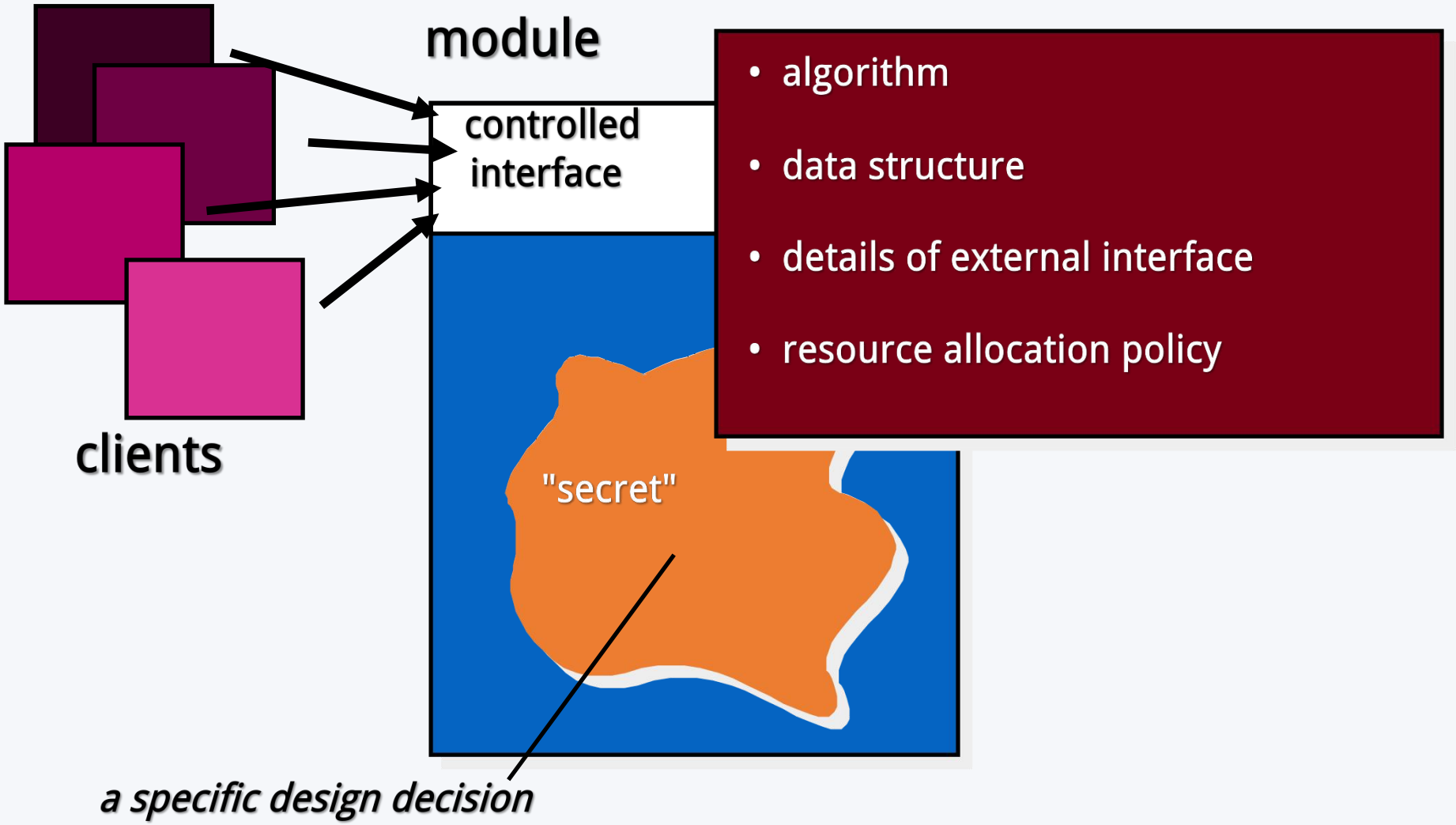
Integration Cost

Integration cost increases with more modules due to more interfaces, communication overhead, and coordination complexity.

Optimal Balance

The optimal number of modules minimizes total cost while considering quality attributes like maintainability and reusability.

Information Hiding



Design Benefits

Why Information Hiding?

Reduces Side Effects

Reduces likelihood of unintended consequences when modifying code

Limits Global Impact

Limits global impact of local design decisions

Controlled Communication

Emphasizes communication through controlled interfaces

Discourages Global Data

Discourages use of global data which can create hidden dependencies

Encapsulation

Leads to encapsulation — an attribute of high quality design

Higher Quality

Results in higher quality software that is easier to maintain and evolve

Core Principle: Information hiding is fundamental to good software design. It enables modular development, reduces complexity, and creates systems that can evolve gracefully over time.

Design Quality Metrics

Functional Independence

C

Cohesion

(Single-Minded Function)

A measure of how closely related the internal elements of a module are to each other.

Functional Cohesion
All elements contribute to a single, well-defined task

Sequential Cohesion
Output of one element serves as input to the next

Communicational Cohesion
Elements operate on the same input or output data

Goal: High cohesion

L

Coupling

(Low Coupling)

A measure of the degree of interdependence between modules.

Data Coupling
Modules communicate through simple data parameters

Stamp Coupling
Modules share a composite data structure

Control Coupling
One module controls the behavior of another

Goal: Low coupling

Functional Independence

Modules with high cohesion and low coupling are functionally independent, making them easier to understand, maintain, test, and reuse.

Design Quality Metrics

Functional Independence

```
def submit_order(order_details):
    # 1. 扣减库存（数据库操作）
    stock = get_stock(order_details['product_id'])
    if stock <= 0:
        return "没库存了"
    # 2. 检查优惠券（业务逻辑）
    if order_details['coupon_code'] == 'DISCOUNT50':
        total_price = order_details['price'] * 0.5
    # 3. 发送短信通知（外部接口调用）
    send_sms(order_details['user_phone'], "您下单了")
    # 4. 写日志（I/O 操作）
    log_file.write("订单已提交")
```



```
# 仅通过简单的数据参数交互（数据耦合）
inventory.deductStock(product_id, 1)  # 传两个简单的字符串/数字
final_price = coupon.calculate_final_price(price, coupon_id)
notification.notify(user_id, "下单成功")
```

System Structure

Architecture

Structural Properties

Defines the components of a system and how they are packaged and interact.

Components

- Modules
- Objects
- Filters

Example: Objects encapsulate data and processing, interacting via methods

Extra-Functional Properties

Addresses how the architecture meets system characteristics beyond core functionality.

- **Performance**
- **Capacity**
- **Reliability**
- **Security**
- **Adaptability**
- **Scalability**

Families of Systems

Architecture should draw upon repeatable patterns for similar systems.

Reusability

Ability to reuse architectural building blocks

Domain Patterns

Common solutions for specific domains

Product Lines

Shared architecture across related products

Architecture Definition

Software architecture encompasses the set of significant decisions about the organization of a software system including structural elements, their externally visible properties, and the relationships between them.

Design Improvement

Refactoring

Fowler [FOW99] defines refactoring as changing a system's internal structure without altering external behavior

Definition

"Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."

— Martin Fowler, 1999

What to Examine in Refactoring

Redundancy
Duplicated code that can be consolidated

Unused Elements
Dead code, unused variables, or obsolete components

Inefficient Algorithms
Poorly performing or unnecessary algorithms

Poor Data Structures
Inappropriate or inefficient data organization

Design Failures
Any design issues that can be corrected for better design

Refactoring Process

Refactoring is a continuous process of improving design quality. It should be done in small steps, with testing after each change to ensure behavior preservation. Good refactoring leads to better maintainability, readability, and extensibility.

Object-Oriented Design

OO Design Concepts

Design Classes

User Interface Classes

Define user interactions and presentation

Business Domain Classes

Represent business entities and rules

Process Classes

Implement business processes and workflows

Persistent Classes

Manage long-term data storage

System Classes

Provide system-level services

Inheritance

All responsibilities of a superclass are immediately inherited by all subclasses, promoting code reuse.

Messages

Stimulate behavior to occur in the receiving object, enabling object communication.

Polymorphism

A characteristic that greatly reduces effort required to extend the design.

OO Design Options

Inheritance

Four approaches to creating new classes

1 From Scratch

The class can be designed and built from scratch. Inheritance is not used.

When: No suitable superclass exists, or creating a completely new abstraction

2 Inherit from Superclass

Search hierarchy for a class higher in the hierarchy. Inherit and add as required.

When: A superclass contains most required attributes and operations

3 Restructure Hierarchy

Restructure hierarchy so that required attributes can be inherited by the new class.

When: Existing hierarchy doesn't quite fit, but refactoring enables reuse

4 Override Characteristics

Override existing characteristics and implement different versions for the new class.

When: Need similar but specialized behavior from an existing class

Inheritance Design Principle

Inheritance should promote code reuse while maintaining a clear "is-a" relationship. Choose the approach that balances reuse with proper abstraction and maintainability.

Object Communication

Messages

Definition

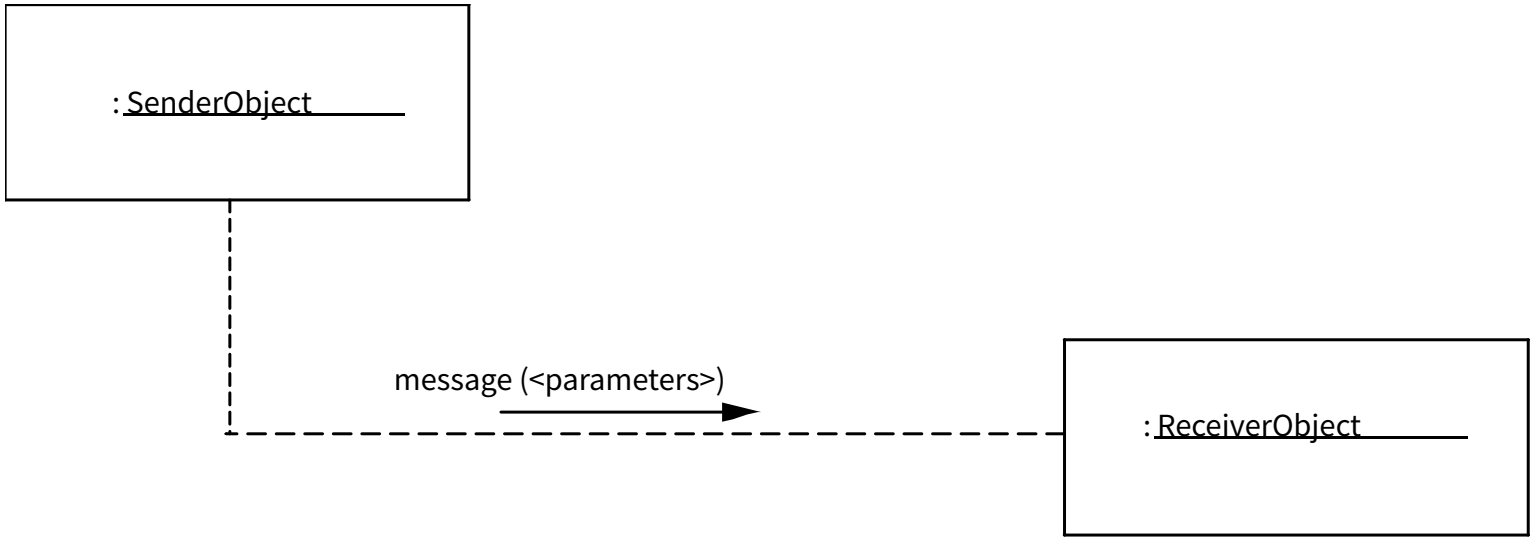
Messages stimulate behavior to occur in the receiving object.

Communication

The primary communication mechanism between objects.

Implementation

Implemented as method calls in most OO languages.



Key Principle

Messages enable loose coupling between objects. The sender only needs to know the interface, not the internal implementation of the receiver.

OO Flexibility

Polymorphism

Many forms, one interface

Conventional Approach

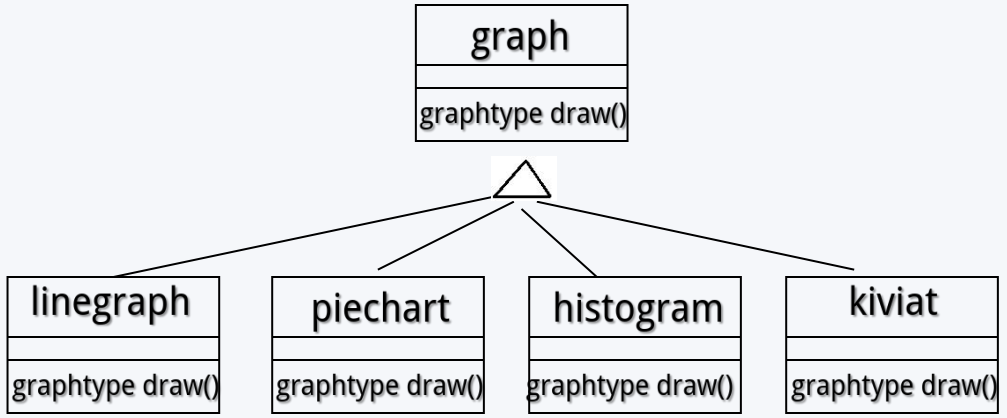
```
case of graphtype:
  if graphtype = linegraph then
    DrawLineGraph (data);
  if graphtype = piechart then
    DrawPieChart (data);
  if graphtype = histogram then
    DrawHisto (data);
  if graphtype = kiviat then
    DrawKiviat (data);
end case;
```

Problems: Complex conditional logic, hard to extend, tightly coupled

Polymorphism Approach

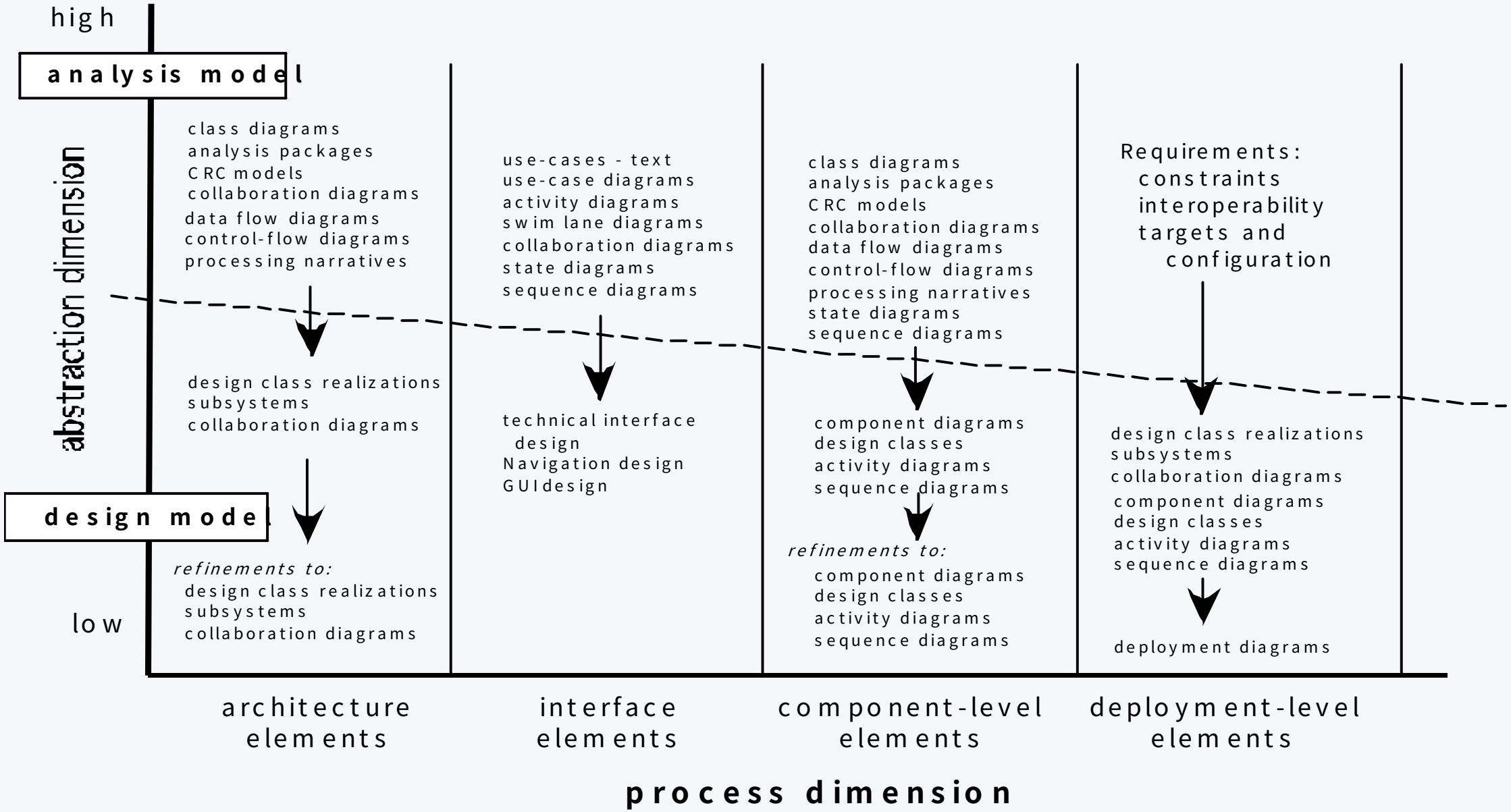
All graphs become subclasses of a general class called graph.
Each subclass defines an operation called draw.
An object sends a draw message to any object instantiated from any subclass.
The receiving object invokes its own draw operation.

Benefits: Simple, extensible, loosely coupled, easy to add new graph types



Design Representation

The Design Model



Model Components

Design Model Elements

Data Design

Data Model →
Data Structures

Data Model →
Database Architecture

Architectural Design

Derived from
Application Domain

Analysis Classes →
Design Realizations

Patterns & Styles

Interface Design

User Interface (UI)
Usability Design

External Interfaces

Internal Interfaces

Component Elements

Component-level design specifications

Deployment Elements

Physical distribution specifications

Integrated Design

All these elements work together to create a complete design model that addresses every aspect of the software system from data to deployment.

Interface Elements

Interface is a specifier for externally-visible operation of a class, component, or classifier.

Key Points

- ✓ Specifies what, not how
- ✓ No internal structure details
- ✓ Focuses on behavior specification

Interface is a set of operations describing behavior and providing access.

Characteristics

- ✓ Describes part of behavior
- ✓ Provides access to operations
- ✓ Contract for implementation

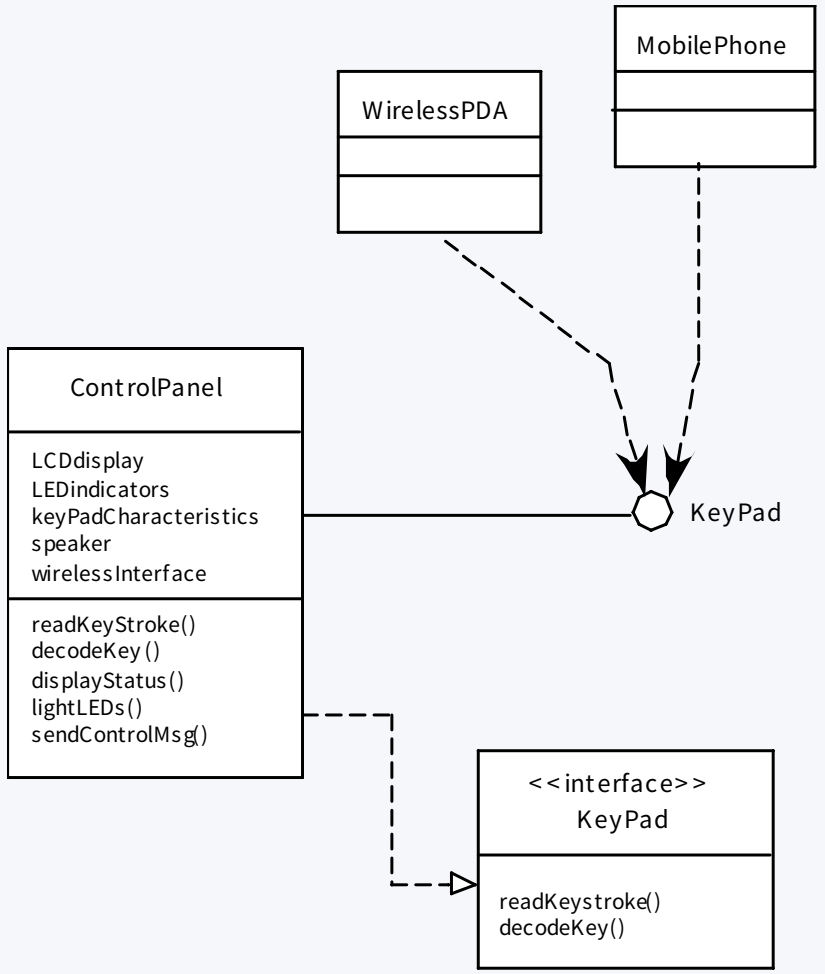


Figure 9.6 UML interface representation for **ControlPanel**

Interface Design

Interface Elements

```java
public interface Switchable {
    void turnOn();    // 打开
    void turnOff();   // 关闭
}
```

```java
public class SmartLight implements Switchable {
    private int brightness; // 内部私有属性
    public void turnOn() {
        System.out.println("灯光亮起，亮度 80%");
    }
    public void turnOff() {
        System.out.println("灯光关闭");
    }
}
```

```java
public class SmartCurtain implements Switchable {
    public void turnOn() {
        System.out.println("窗帘电机正转，窗帘打开");
    }
    public void turnOff() {
        System.out.println("窗帘电机反转，窗帘关闭");
    }
}
```

```java
public class ControlPanel {
    // 它只依赖 Switchable 接口，根本不认识具体的 SmartLight 类
    public void control(Switchable device) {
        device.turnOn();  // 发送消息
        device.turnOff(); // 发送消息
    }
}
```

Component Elements

Component-Level Design

Component elements represent the detailed specifications of individual software components.

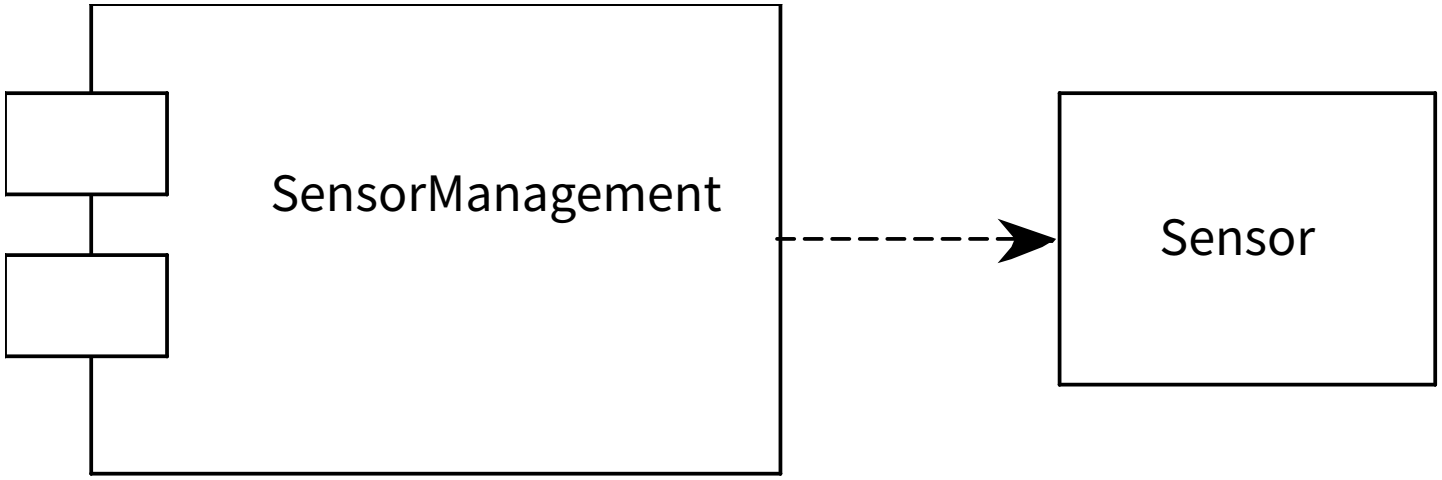
Component Specification

- Interface definitions
- Data structures
- Algorithm details
- Dependencies

Design Objectives

- High cohesion
- Low coupling
- Reusability
- Testability

Component Relationships



Component Design Principle

Components should be designed as independent, replaceable units with well-defined interfaces. This enables modular development, testing, and maintenance.

Deployment Elements

Deployment Design

Deployment elements specify the physical distribution of software components across hardware nodes.

Deployment Considerations

- ✓ Hardware allocation
- ✓ Network topology
- ✓ Communication protocols
- ✓ Performance requirements

Deployment Diagram

Shows nodes, artifacts, and their relationships

Deployment Architecture

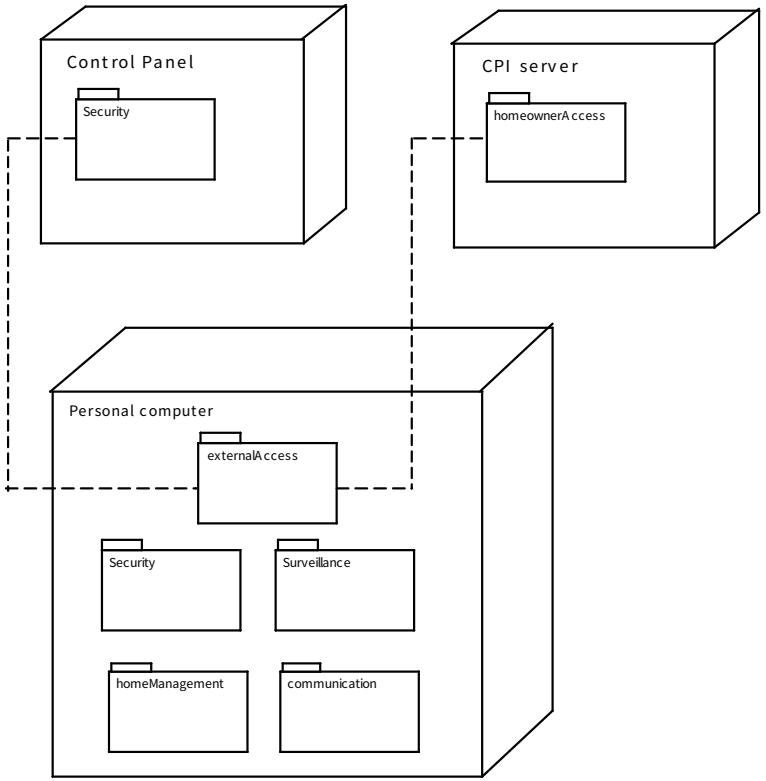


Figure 9.8 UML deployment diagram for *SafeHome*

Deployment Strategy

Deployment design must consider both functional requirements and quality attributes like performance, scalability, and reliability. The physical architecture should support current needs while allowing for future evolution.

Knowledge Check

Exercise - Questions 1-4

1. Which of the following are areas of concern in the design model? Answer: e
    a. architecture
    b. data
    c. interface
    d. project scope
    e. a, b and c
2. Which of these are characteristics of a good design? Answer: e
    a. exhibits strong coupling between its modules
    b. implements all requirements in the analysis model
    c. includes test cases for all components
    d. provides a complete picture of the software
    e. both b and d
3. Information hiding makes program maintenance easier by hiding data and procedure from unaffected parts of the program. Answer: a
    a. True
    b. False
4. Cohesion is a qualitative indication of the degree to which a module Answer: b
    a. can be written more compactly.
    b. focuses on just one thing.
    c. is able to complete its function in a timely manner.
    d. is connected to other modules and the outside world.

Knowledge Check

Exercise - Questions 5-8

5. Coupling is a qualitative indication of the degree to which a module Answer: d
    a. can be written more compactly.
    b. focuses on just one thing.
    c. is able to complete its function in a timely manner.
    d. is connected to other modules and the outside world.
6. Polymorphism reduces the effort required to extend an object system by Answer: b
    a. coupling objects together more tightly.
    b. enabling a number of different operations to share the same name
    c. making objects more dependent on one another.
    d. removing the barriers imposed by encapsulation.
7. Which design model elements are used to depict a model of information represented from the user's view? Answer: c
    a. Architectural design elements
    b. Component-level design elements
    c. Data design elements
    d. Interface design elements
8. Which design is analogous to the floor plan of a house? Answer: a
    a. Architectural design elements
    b. Component-level design elements
    c. Data design elements
    d. Interface design elements

Knowledge Check

Exercise - Questions 9-12

9. Which design model is analogous to the detailed drawings of the access points and external utilities for a house?
    a. Architectural design elements
    b. Component-level design elements
    c. Data design elements
    d. Interface design elements
    Answer: d
10. Which design model is analogous to a set of detailed drawings for each room in a house?
    a. Architectural design elements
    b. Component-level design elements
    c. Data design elements
    d. Interface design elements
    Answer: b
11. The deployment design elements specify the build order for the software components.
    a. True
    b. False
    Answer: b
12. One of the key problems in software reuse is the inability to find existing reusable design patterns when hundreds of candidates exist.
    a. True
    b. False
    Answer: a